

LAPORAN PRAKTIKUM

MODUL 9: JAVASCRIPT LANJUTAN

GIK1JAB3 PEMROGRAMAN WEB



Oleh:

Kelas : D4 SIKC-49-03

NIM	NAMA
707012500097	Felda Muffarihati Azizah

PROGRAM STUDI S.Tr SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

A. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep dasar JavaScript sebagai bahasa pemrograman web.
2. Memahami fitur Modern JavaScript (ES6+) dan perannya sebagai fondasi React.
3. Mampu menggunakan sintaks ES6+ secara tepat dan konsisten.
4. Memahami konsep state, immutability, dan functional programming.
5. Mampu membangun aplikasi sederhana berbasis state menggunakan Vanilla JavaScript.

B. Alat dan Bahan

Untuk menjalankan modul ini, diperlukan komputer atau laptop dengan sistem operasi Windows, Linux, atau macOS, minimal prosesor 1 GHz, RAM 2GB (disarankan 4GB atau lebih), serta browser modern seperti Google Chrome, Mozilla Firefox, atau Microsoft Edge. Perangkat lunak yang digunakan meliputi:

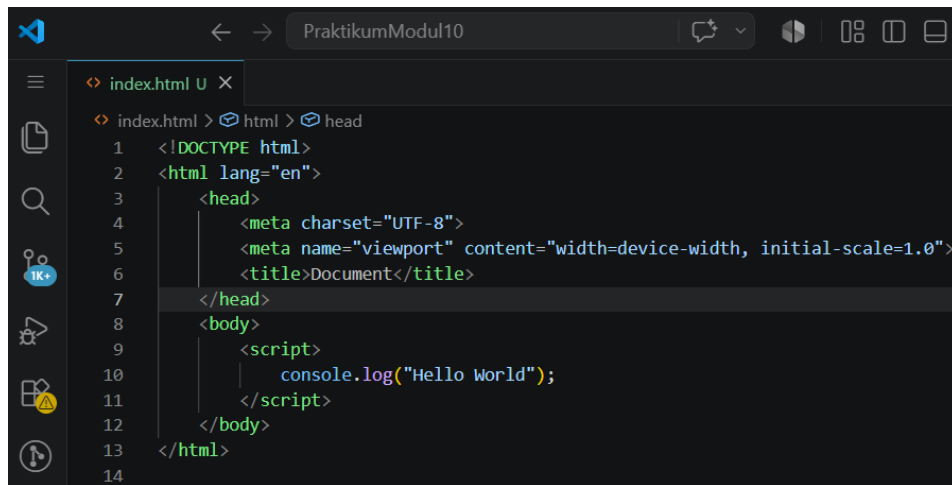
- Code editor (Visual Studio Code, Sublime Text, atau sejenisnya)
- Web browser

Bahan yang dibutuhkan:

- File HTML dasar
- File JavaScript (.js)

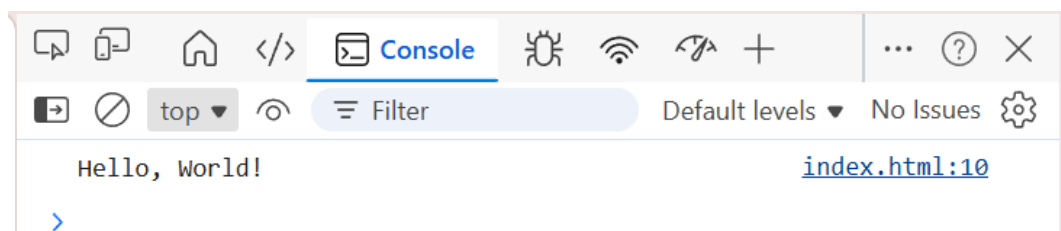
C. Langkah-Langkah Praktikum dan Dokumentasi Program

Buatlah sebuah file baru dengan nama index.html menggunakan Visual Studio Code. Pada file tersebut, ketik tanda seru ! atau ketik html:5, kemudian tekan tombol Enter. Perintah ini akan menghasilkan struktur dasar (template) HTML secara otomatis.



```
index.html U X
index.html > html > head
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Document</title>
7   </head>
8   <body>
9     <script>
10      console.log("Hello World");
11    </script>
12  </body>
13 </html>
14
```

Tulisan "Hello JavaScript" akan muncul pada console browser ketika halaman dijalankan.

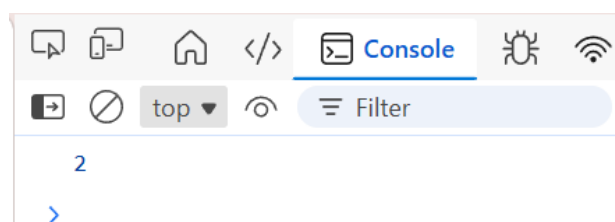


Praktikum 1 (penggunaan Let)

Buat file JavaScript baru dengan nama let-const.js. Tuliskan kode berikut:

```
JS let-const.js > ...
1 //Praktikum 1 (penggunaan Let)
2 let counter = 1
3 counter = 2
4 console.log(counter)
```

Hubungkan file let-const.js ke dalam index.html menggunakan tag `<script>`. Jalankan index.html, lalu buka Inspect → Console.

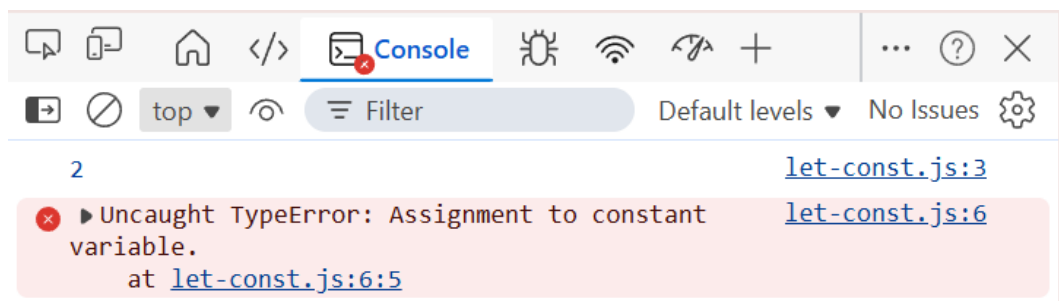


Praktikum 2 (Penggunaan const)

Tambahkan kode berikut ke dalam file let-const.js

```
7 //Praktikum 2 (Penggunaan const)
8 const MAX = 10;
9 MAX = 20; // error: Assignment to constant variable.
```

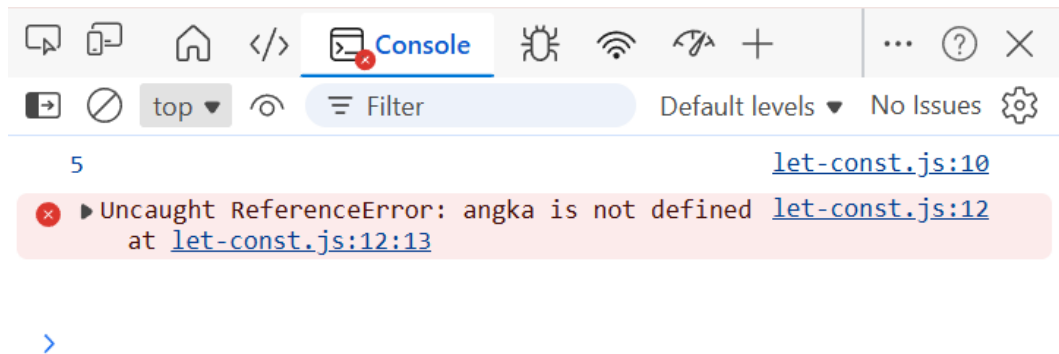
Refresh halaman browser dan buka inspect -> console. Console menampilkan error karena nilai const tidak dapat diubah.



Praktikum 3 (Contoh block scope)

Tambahkan kode if tersebut ke dalam file let-const.js. Variabel angka hanya dapat diakses di dalam blok if. Ketika diakses di luar blok, JavaScript akan menampilkan error. Hal ini membuktikan bahwa let memiliki cakupan blok (block scope).

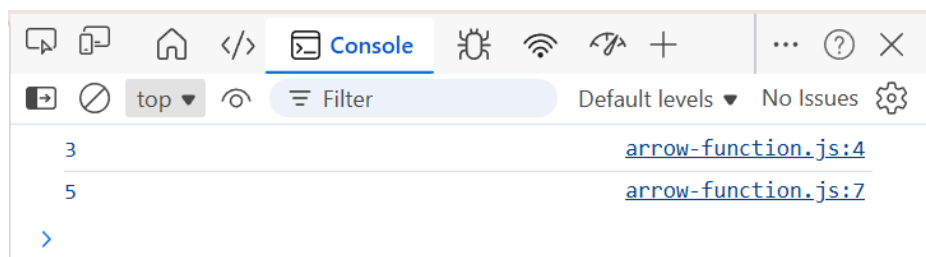
```
12 //Praktikum 3 (Contoh block scope) :
13 if (true) {
14     let angka = 5;
15     console.log(angka);
16 }
17 console.log(angka); // error: angka is not defined
```



Praktikum 4 (Perbandingan Function Biasa dan Arrow Function)

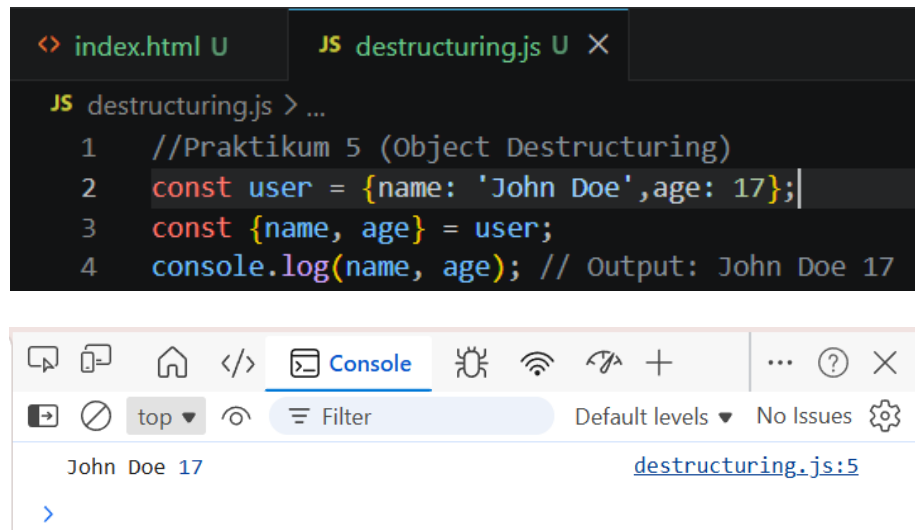
Buat file JavaScript baru dengan nama arrow-function.js. Tuliskan fungsi biasa berikut. Tambahkan arrow function di bawahnya. Hubungkan file arrow-function.js ke dalam index.html.

```
JS arrow-function.js > ...
1  function tambah(a, b) {
2    |    return a + b;
3  }
4  console.log(tambah(1, 2)); // Output: 3
5
6  const tambahArrow = (a, b) => a + b;
7  console.log(tambahArrow(2, 3)); // Output: 5
```



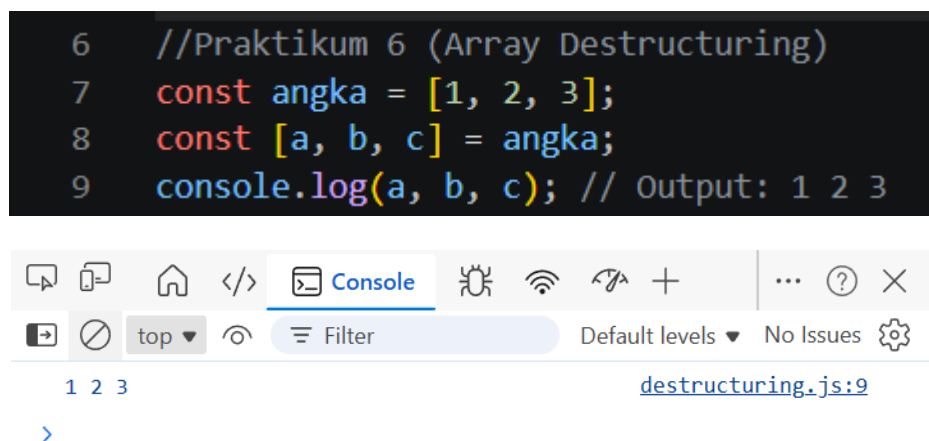
Praktikum 5 (Object Destructuring)

Buat file JavaScript baru dengan nama destructuring.js. Masukkan kode berikut ke dalam file tersebut. Hubungkan file destructuring.js ke dalam file index.html. Jalankan file HTML dan buka menu Inspect → Console.



Praktikum 6 (Array Destructuring)

Tambahkan kode berikut ke dalam file destructuring.js. dan nonaktifkan kode sebelumnya dengan menambahkan komentar “//”. Refresh halaman browser dan perhatikan console. Console browser akan menampilkan nilai 1, 2, 3.



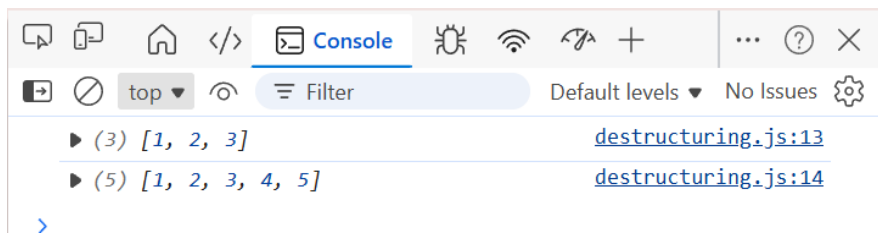
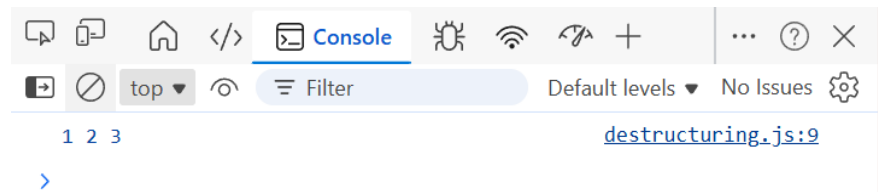
Praktikum 7 (Spread Operator pada Array)

Tambahkan kode berikut ke dalam file destructuring.js. Refresh halaman browser dan buka menu Inspect → Console. Hasil :

- Array data tetap berisi [1, 2, 3]
- Array dataBaru berisi [1, 2, 3, 4]

Hal ini menunjukkan bahwa spread operator membuat salinan data baru tanpa mengubah data asli.

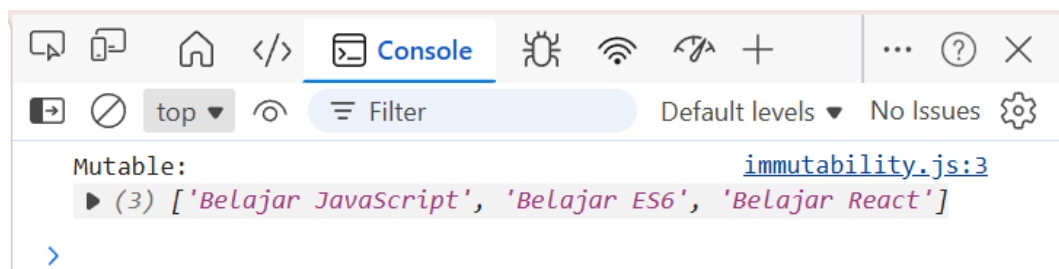
```
11 //Praktikum 7 (Spread Operator)
12 const data = [1, 2, 3];
13 const dataBaru = [...data, 4, 5];
14 console.log(data); // Output: [1, 2, 3]
15 console.log(dataBaru); // Output: [1, 2, 3, 4, 5]
```



Praktikum 8 – Immutability pada Array

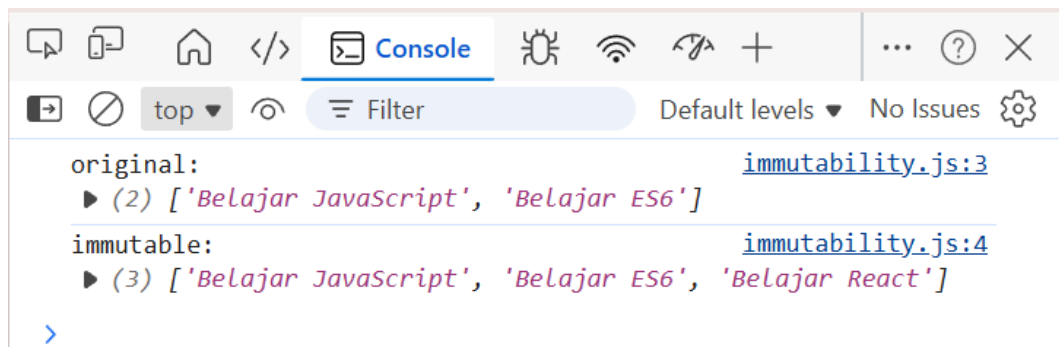
Buat file JavaScript baru dengan nama immutability.js. Masukkan kode berikut: Hubungkan file immutability.js ke dalam file index.html, jalankan file HTML dan buka Inspect → Console.

```
JS immutability.js > ...
1 //Praktikum 8 - Immutability pada Array
2 let tasks = ['Belajar JavaScript', 'Belajar ES6'];
3
4 tasks.push('Belajar React'); // Mengubah array tasks secara langsung
5 console.log("Mutable:", tasks);
```



Ubah Kode Sebelumnya menjadi:

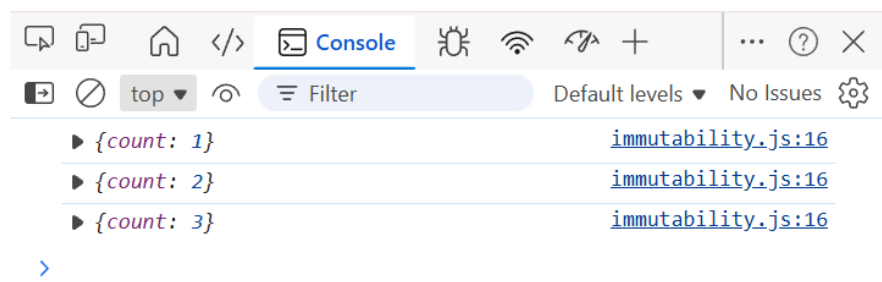
```
7 let tasks = ['Belajar JavaScript', 'Belajar ES6'];
8 let newTasks = [...tasks, 'Belajar React'];
9 console.log("original:", tasks);
10 console.log("immutable:", newTasks);
```



Praktikum 9 Simulasi State-Oriented Programming (Vanilla JS)

Tambahkan kode berikut ke dalam file `immutability.js`

```
12 let state = {
13   count: 0,
14 };
15
16 function increment() {
17   state = {
18     ... state,
19     count: state.count + 1,
20   };
21
22   console.log(state);
23 }
24 increment();
25
26 increment();
27
28 increment();
```

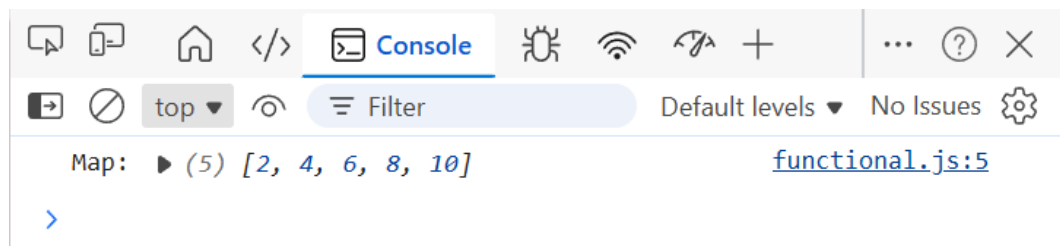


Praktikum 10 map()

Buat file JavaScript baru dengan nama functional.js, masukkan kode berikut:

```
JS functional.js > ...
1 //Praktikum 10 map()
2 const number = [1, 2, 3, 4, 5];
3 const hasilMap = number.map(function(n) {
4   |   return n * 2;
5   | });
6 console.log("Map:", hasilMap); // Output: [2, 4, 6, 8, 10]
```

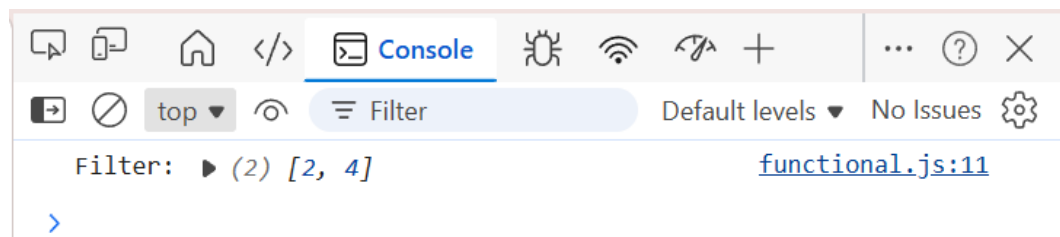
Hubungkan file functional.js ke dalam index.html. Jalankan file HTML dan buka Inspect → Console. Hasil : Map: [2, 4, 6, 8, 10]



Praktikum 11 filter()

Tambahkan kode berikut ke dalam file functional.js

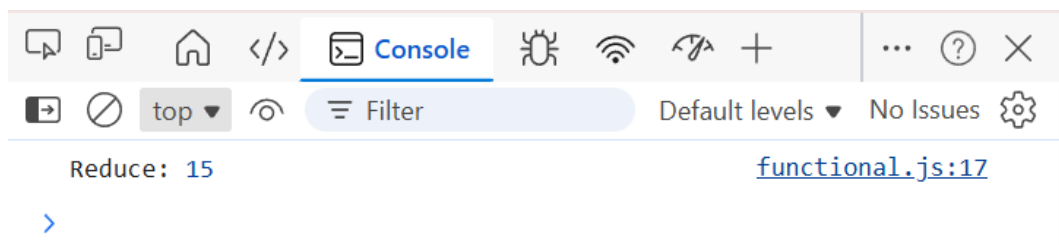
```
8 //Praktikum 11 filter()
9 const number = [1, 2, 3, 4, 5];
10 const hasilFilter = number.filter(function(n) {
11   |   return n % 2 === 0;
12   | });
13 console.log("Filter:", hasilFilter); // Output: [2, 4]
```



Praktikum 12 reduce()

Tambahkan kode berikut ke dalam file functional.js

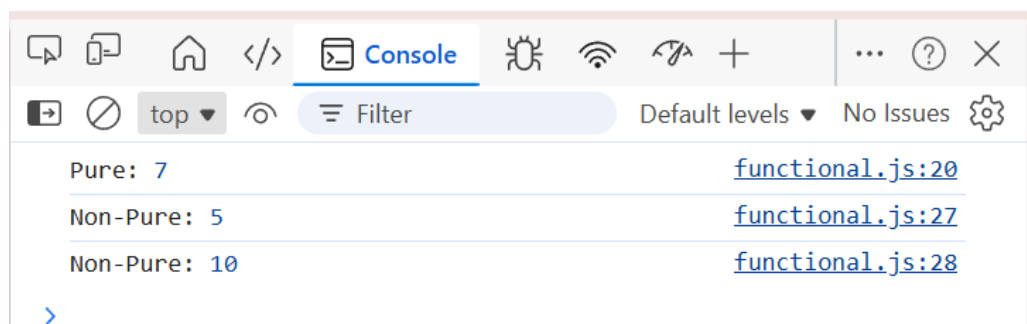
```
15 //Praktikum 12 reduce()
16 const number = [1, 2, 3, 4, 5];
17 const hasilReduce = number.reduce(function(acc, n) {
18   |   return acc + n;
19   | }, 0);
20 console.log("Reduce:", hasilReduce); // Output: 15
```



Praktikum 13 Pure vs Non-Pure Function

Tambahkan kode berikut ke dalam file functional.js.

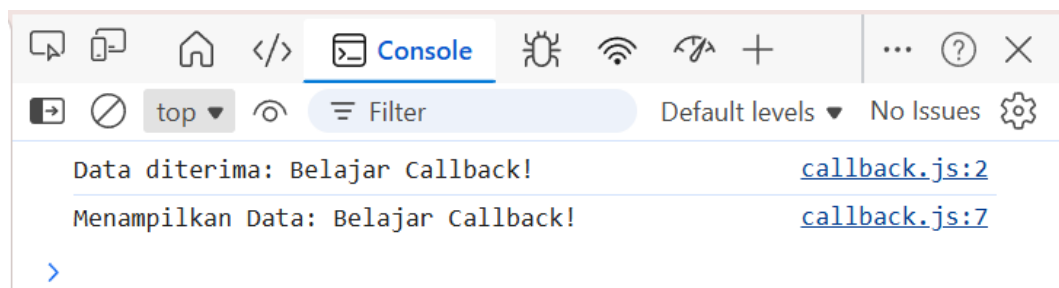
```
22 //Praktikum 13 (Pure vs Non-Pure Functions)
23 const tambah = (a, b) => a + b;
24 console.log("Pure:", tambah(3, 4)); // Output: 7
25
26 let nilai = 0;
27 function tambahGlobal(x) {
28   |   nilai += x;
29   |   return nilai;
30   | }
31 console.log("Non-Pure:", tambahGlobal(5)); // Output: 5
32 console.log("Non-Pure:", tambahGlobal(5)); // Output: 10
```



Praktikum 14 Callback Dasar

Buat file JavaScript baru dengan nama callback.js, masukkan kode berikut dan hubungkan file callback.js ke dalam index.html

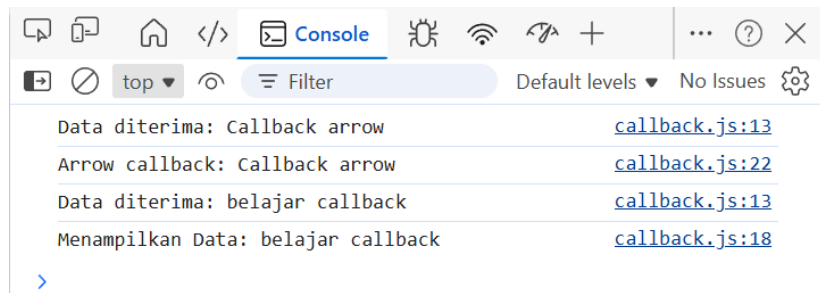
```
JS callback.js >  tampilkanData
1 //Praktikum 14 - Callback Dasar
2 function prosesData(data, callback) {
3     console.log("Data diterima:", data);
4     callback(data);
5 }
6 function tampilkanData(data) {
7     console.log("Menampilkan Data:", data);
8 }
9 prosesData("Belajar Callback!", tampilkanData);
```



Praktikum 15 (Callback dengan Arrow Function)

Tambahkan kode berikut ke dalam callback.js, refresh browser dan cek inspect->console

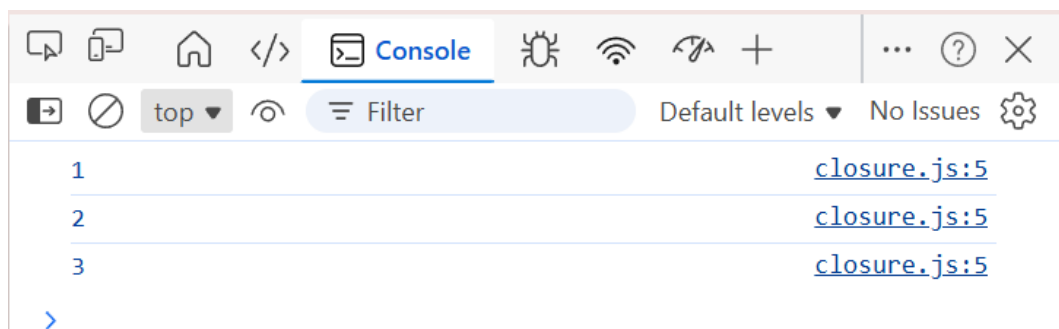
```
12 //Praktikum 15 - Callback dengan Arrow Function
13 function prosesData(data, callback) {
14     console.log("Data diterima:", data);
15     callback(data);
16 }
17 function tampilkanData(data) {
18     console.log("Menampilkan Data:", data);
19 }
20 prosesData("Callback arrow", (data) => {
21     console.log("Arrow callback:", data);
22 });
23 prosesData("belajar callback", tampilkanData);
```



Praktikum 16 Closure Dasar

Buat file JavaScript baru dengan nama closure.js, masukkan kode berikut:

```
JS closure.js >  buatState
1 //Praktikum 16 Closure Dasar
2 function buatCounter() {
3   let coun = 0;
4   return function() {
5     coun++;
6     console.log(coun);
7   };
8 }
9 const counter = buatCounter();
10 counter(); // Output: 1
11 counter(); // Output: 2
12 counter(); // Output: 3
```



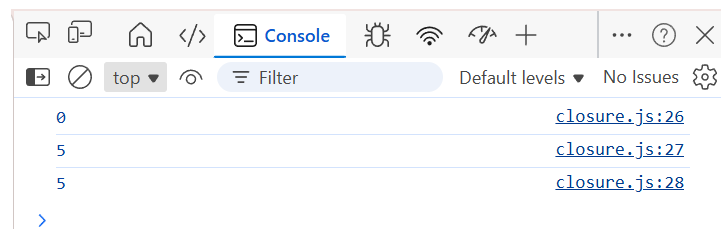
Praktikum 17 Closure sebagai State

Tambahkan kode berikut ke dalam closure.js

```

15 //Praktikum 17 Closure sebagai State
16 function buatState(nilaiAwal) {
17     let state = nilaiAwal;
18     return function(nilaiBaru) {
19         if (nilaiBaru !== undefined) {
20             state = nilaiBaru;
21         }
22         return state;
23     };
24 }
25 const hitung = buatState(0);
26 console.log(hitung());
27 console.log(hitung(5));
28 console.log(hitung());

```



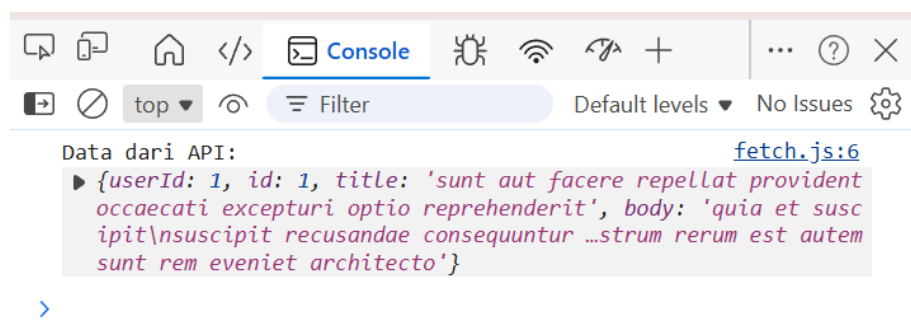
Praktikum 18 Fetch Data dari API

Buat file JavaScript baru dengan nama fetch.js

```

JS fetch.js > ...
1 //Praktikum 18 Fetch Data dari API
2 fetch("https://jsonplaceholder.typicode.com/posts/1")
3   .then(function(response){
4     return response.json();
5   })
6   .then(function(data){
7     console.log("Data dari API:", data);
8   });
9

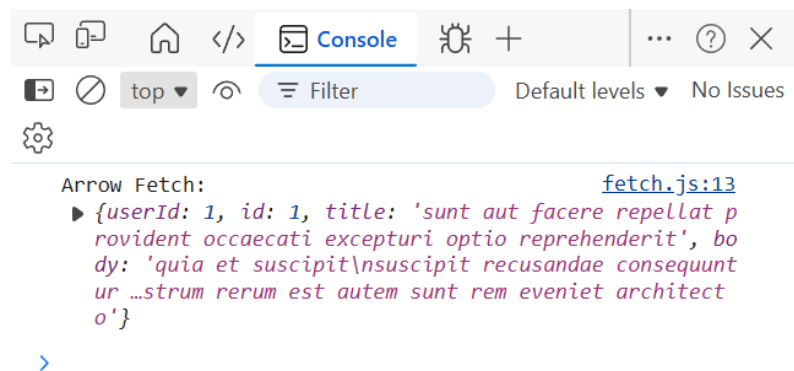
```



Praktikum 19 Fetch dengan Arrow Function

Ubah isi file fetch.js menjadi:

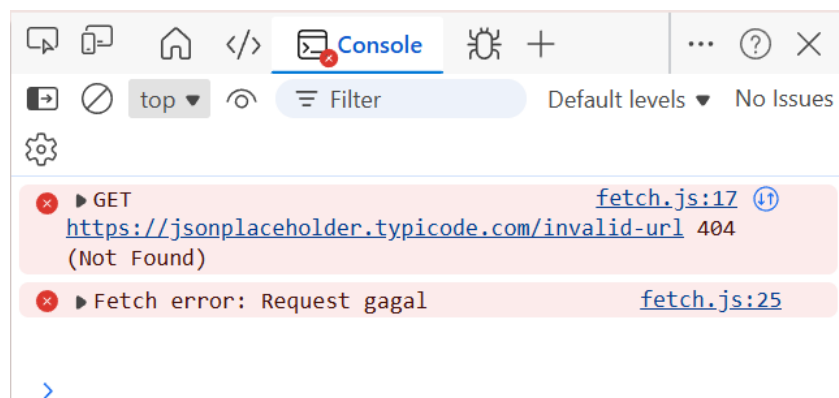
```
10 //PRAKTIKUM 19 FETCH ARROW FUNCTION
11 fetch("https://jsonplaceholder.typicode.com/posts/1")
12   .then(res => res.json())
13   .then(data => console.log("Arrow Fetch:", data));
14
```



Praktikum 20 – Menangani Error Fetch

Ubah isi file fetch.js menjadi:

```
16 //PRAKTIKUM 20 Error Fetch
17 fetch("https://jsonplaceholder.typicode.com/invalid-url")
18   .then(res => {
19     if (!res.ok) {
20       throw new Error("Request gagal");
21     }
22     return res.json();
23   })
24   .then((data) => console.log(data))
25   .catch((error) => console.error("Fetch error:", error.message));
```



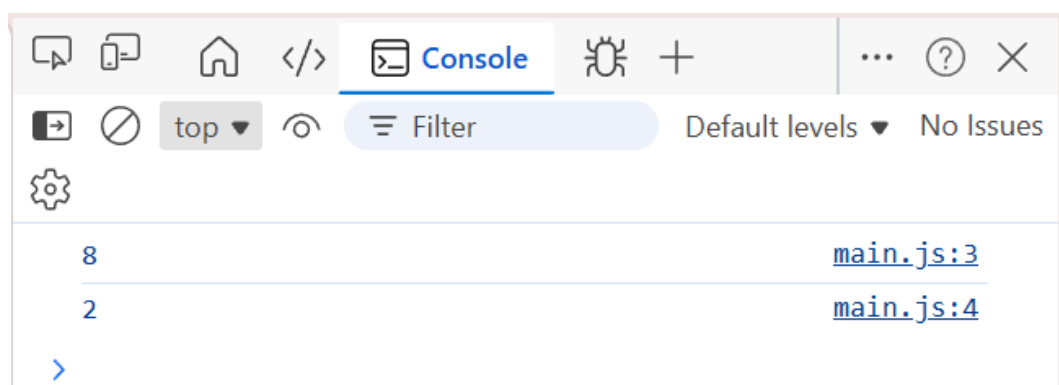
Praktikum 21 – Module Dasar (export & import)

Buat file JavaScript baru dengan nama `utils.js`, isi file tersebut dengan kode berikut:

```
JS utils.js > tambah
1 //Praktikum 21 – Module Dasar (export & import)
2 export function tambah (a, b) {
3   return a + b;
4 }
5 export function kurang (a, b) {
6   return a - b;
7 }
```

Buat file JavaScript baru dengan nama `main.js`, isi file tersebut dengan kode berikut:

```
JS main.js
1 //Praktikum 21 – Module Dasar (export & import)
2 import {tambah, kurang} from './utils.js';
3
4 console.log(tambah(5, 3));
5 console.log(kurang(5, 3));
```



Praktikum 22 – Component Sederhana (Vanilla JS)

Buat file JavaScript baru dengan nama `counterComponent.js`, masukkan kode berikut:

```

JS counterComponent.js > ...
1  //Praktikum 22 - Component Sederhana (Vanilla JS)
2  export function Counter() {
3      let count = 0;
4
5      function increment() {
6          count++;
7          render();
8      }
9
10     function render() {
11         document.getElementById("app").innerHTML = `
12         <p>count: ${count}</p>
13         <button id="btn">Tambah</button>
14         `;
15
16         document.getElementById("btn").onclick = increment;
17     }
18     render();
19 }

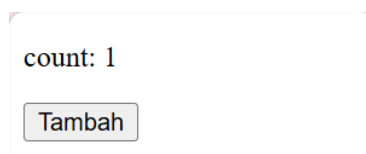
```

Ubah isi main.js menjadi:

```

7  //Praktikum 22 - Component Sederhana (Vanilla JS)
8  import { Counter } from './counterComponent.js';
9  Counter();

```



Praktikum 23 – Component Reusable

Tambahkan parameter dan tag<h3>{title}</h3> pada file counterComponent.js:

```

21  //Praktikum 23 - Component Reusable
22  export function Counter(title) {
23      let count = 0;
24
25      function increment() {
26          count++;
27          render();
28      }
29
30      function render() {
31          document.getElementById("app").innerHTML = `
32          <h1>${title}</h1>
33          <p>count: ${count}</p>
34          <button id="btn">Tambah</button>
35          `;
36
37          document.getElementById("btn").onclick = increment;
38      }
39      render();
40  }

```


Ubah file main.js

```
11 //Praktikum 23 - Component Reusable
12 import { Counter } from './counterComponent.js';
13
14 Counter("Counter Pertama");
```

Counter Pertama

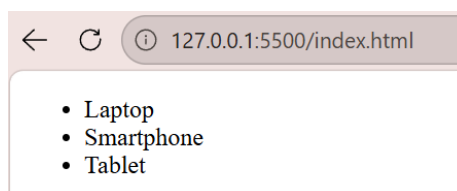
count: 0

Tambah

Praktikum 24 – Menampilkan Data Array ke HTML

Buat file baru bernama render.js

```
JS render.js > ...
1 //Praktikum 24 - Menampilkan Data Array ke HTML
2 const products = ["Laptop", "Smartphone", "Tablet"];
3 const app = document.getElementById("app");
4 app.innerHTML = `
5     <ul>
6         <li>${products[0]}</li>
7         <li>${products[1]}</li>
8         <li>${products[2]}</li>
9     </ul>
10 `;
```



Ganti seluruh isi render.js menjadi:

```
12 //Praktikum 24 - Menampilkan Data Array ke HTML dengan Looping
13 const products = ["Laptop", "Smartphone", "Tablet"];
14 const app = document.getElementById("app");
15 const listHTML = products
16     .map(item => `<li>${item}</li>`)
17     .join("");
18 app.innerHTML = `<ul>${listHTML}</ul>`;
```

List produk tetap tampil, namun kode lebih ringkas dan scalable

Praktikum 25 – Menampilkan Data Object ke HTML

Ubah data di render.js menjadi:

```
21 //Praktikum 25 – Menampilkan Data Object ke HTML
22 const products = [
23   { name: "Laptop", price: 1000 },
24   { name: "Smartphone", price: 500 },
25   { name: "Tablet", price: 300 }
26 ];
27
28 const app = document.getElementById("app");
29 const listHTML = products
30   .map(item => `
31     <div>
32       <h4>${item.name}</h4>
33       <p>Harga: $${item.price}</p>
34     </div>
35   `)
36   .join("");
37 app.innerHTML = listHTML;
```

Laptop
Harga: \$1000

Smartphone
Harga: \$500

Tablet
Harga: \$300

Praktikum 26 – Event Handling Klik Dasar

```
JS event.js > ...
1 //Praktikum 26 – Event Handling Klik Dasar
2 const app=document.getElementById("app");
3 app.innerHTML=`
4   <h1>Event Handling</h1>
5   <button id="btn">Klik Saya</button>
6 `;
7 const button=document.getElementById("btnKlik");
8 button.addEventListener("click", function() {
9   alert("Tombol diklik!");
10 });
```



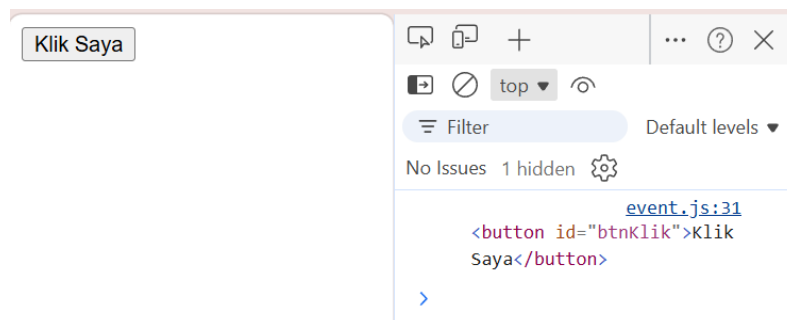
Praktikum 27 – Event Handler Menggunakan Function

```
13 //Praktikum 27 – Event Handler Menggunakan Function
14 const app=document.getElementById("app");
15 app.innerHTML=`
16   <button id="btnKlik">Klik Saya</button>
17 `;
18
19 const button=document.getElementById("btnKlik");
20 function handleClick() {
21   alert("functional handleClick dipanggil");
22 }
23 button.addEventListener("click", handleClick);
```



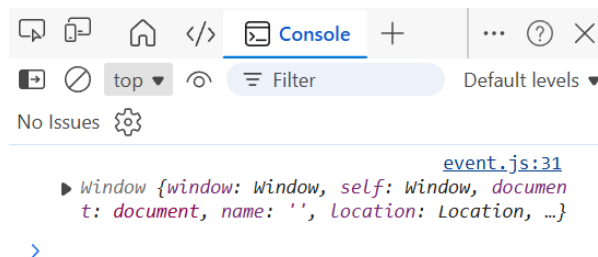
Praktikum 28 – Mengenal this Context

```
26 //Praktikum 28 – Mengenal this Context
27 const app=document.getElementById("app");
28 app.innerHTML=`
29   <button id="btnKlik">Klik Saya</button>
30 `;
31 const button=document.getElementById("btnKlik");
32 button.addEventListener("click", function() {
33   console.log(this); // Akan merujuk pada elemen button
34   alert("Tombol sudah diklik!");
35 });
```



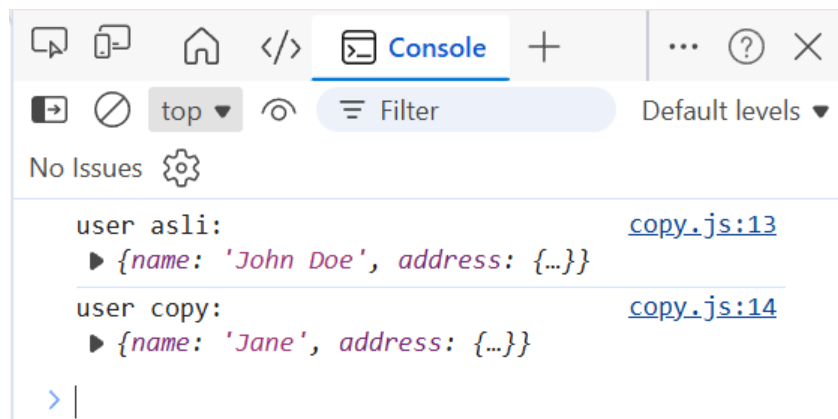
Praktikum 29 – this pada Arrow Function (PERHATIKAN)

```
38 //Praktikum 29 – Event Handler dengan Parameter
39 const app=document.getElementById("app");
40 app.innerHTML=`
41   <button id="btnKlik">Klik Saya</button>
42 `;
43 const button=document.getElementById("btnKlik");
44 button.addEventListener("click", () => {
45   console.log(this);
46 });
```



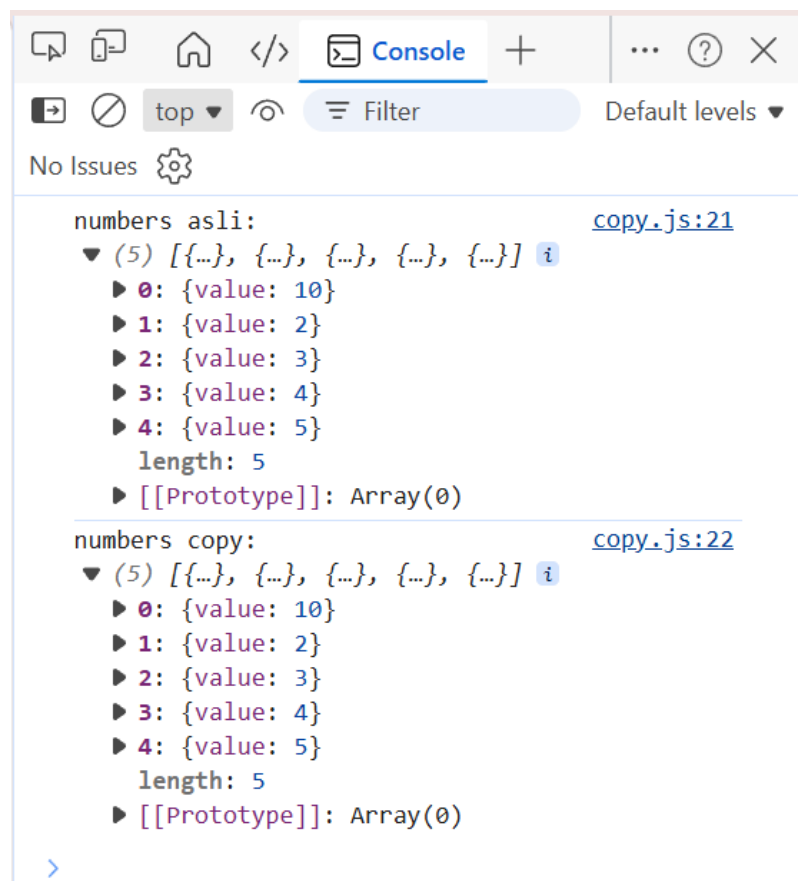
Praktikum 30 – Shallow Copy pada Object

```
JS copy.js > ...
1 //Praktikum 30 – Shallow Copy pada Object
2 const user = {
3   name: "John Doe",
4   address: {
5     street: "jalan Sukabirus Nomor 34",
6     city: "Bandung",
7     country: "Indonesia"
8   }
9 };
10 const userCopy = { ...user };
11 userCopy.name = "Jane";
12 userCopy.address.city = "Jakarta";
13 console.log("user asli:", user);
14 console.log("user copy:", userCopy);
```



Praktikum 31 – Shallow Copy pada Array

```
16 //Praktikum 31 – Shallow Copy pada Array
17 const numbers = [{value: 1}, {value: 2}, {value: 3}, {value: 4}, {value:
18 const numbersCopy = [...numbers];
19
20 numbersCopy[0].value = 10;
21 console.log("numbers asli:", numbers);
22 console.log("numbers copy:", numbersCopy);
23
```



Praktikum 32 – Deep Copy dengan JSON

```
24 //Praktikum 32 – Deep Copy dengan JSON
25 const userDeep = JSON.parse(JSON.stringify(user));
26 userDeep.address.city = "Jakarta";
27 console.log("user asli:", user);
28 console.log("user Deep copy:", userDeep);
```



Praktikum jQuery

Buat file jquery.js (kosong dulu). Buka index.html, isi dengan kode berikut:

```
17 <!DOCTYPE html>
18 <html lang="en">
19 <head>
20   <meta charset="UTF-8">
21   <meta name="viewport" content="width=device-width, initial-scale=1.0">
22   <title>Document</title>
23 </head>
24 <body>
25   <h1 id="judul">Belajar Query</h1>
26   <button id="btn">Klik Saya</button>
27   <div id="app"></div>
28   <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
29   <script src="jquery.js"></script>
30 </body>
31 </html>
```

Tambahkan kode di jquery.js, hasil : judul akan berubah menjadi warna biru

```
JS jquery.js > ...
```

```
1 $("#judul").css("color", "blue");
2
```

Belajar Query

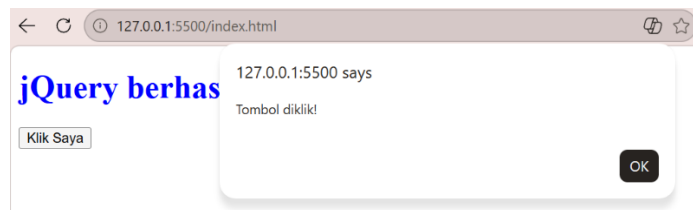
Tambahkan kode untuk merubah teks:

```
JS jquery.js > ...
1  $("#judul").css("color", "blue");
2
3  $("#judul").text("jQuery berhasil jalan");
4
```

jQuery berhasil jalan

Tambahkan kode berikut, klik tombol “Klik Saya” akan memunculkan alert

```
JS jquery.js > ...
1  $("#judul").css("color", "blue");
2
3  $("#judul").text("jQuery berhasil jalan");
4
5  $("#btn").click(function() {
6      alert("Tombol diklik!");
7  });
```



Ganti isi jquery.js seperti kode di bawah untuk merubah warna teks menjadi merah

```
JS jquery.js > ...
1  $("#judul").css("color", "blue");
2
3  $("#judul").text("jQuery berhasil jalan");
4
5  $("#btn").click(function() {
6      alert("Tombol diklik!");
7  });
8
9
10 //F. Event + Manipulasi DOM
11 $("#btn").click(function() {
12     $("#judul").text("Judul berubah karena diklik!");
13     $("#judul").css("color", "red");
14 });
```

Judul berubah karena diklik!

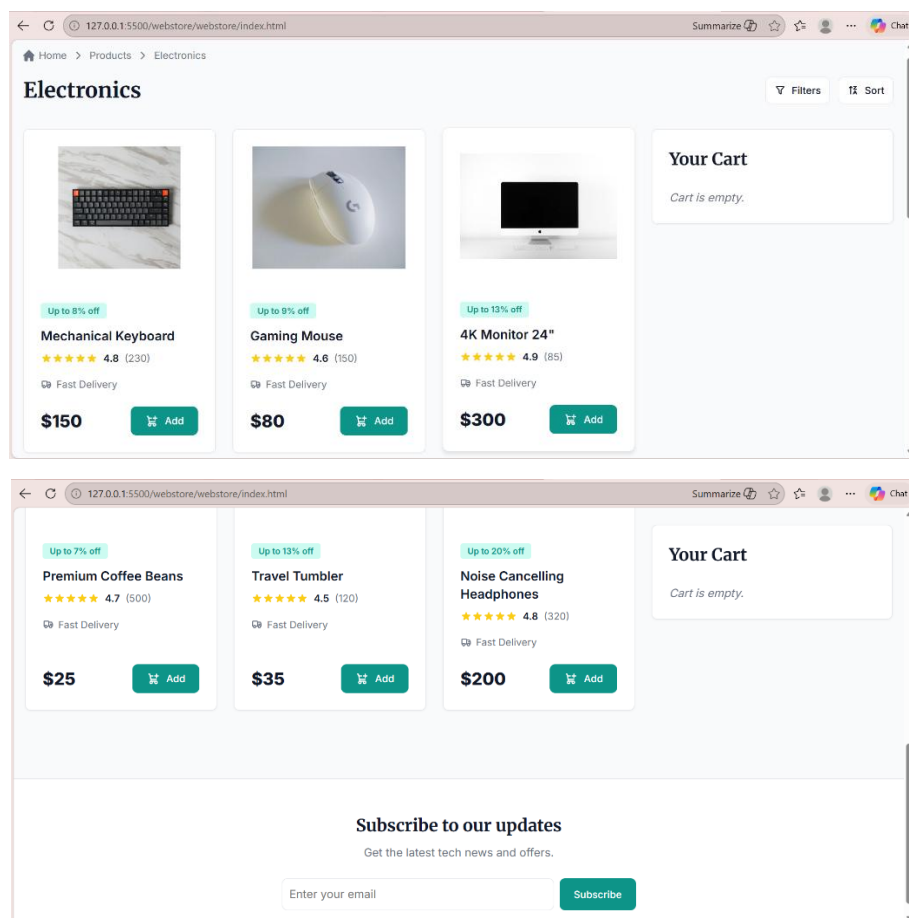
Klik Saya

TUGAS PRAKTIKUM WEBSTORE

Buatlah folder baru bernama webstore dan buat struktur folder seperti dibawah ini :



Kemudia isi file dengan code sesuai dengan yang ada di modul praktikum.



D. Analisis dan Pembahasan

Pada praktikum Modul 10 Modern JavaScript (ES6+), dipelajari konsep JavaScript modern yang digunakan dalam pengembangan web modern dan menjadi dasar framework seperti React. Materi yang dipelajari meliputi penggunaan `let` dan `const`, arrow function, destructuring, spread operator, immutability, functional programming, callback, closure, asynchronous JavaScript, module, hingga component-oriented programming.

Melalui praktikum, dapat dipahami bahwa ES6+ membuat penulisan kode lebih ringkas, terstruktur, dan mudah dikelola. Konsep immutability dan state management juga membantu pengelolaan data menjadi lebih aman tanpa mengubah data asli secara langsung. Selain itu, penggunaan `fetch()` menunjukkan cara mengambil data secara asynchronous dari API.

Pada tugas akhir berupa webstore sederhana, seluruh konsep JavaScript modern diterapkan secara langsung, seperti rendering data produk, event handling, add to cart, state management, modularisasi file, serta penggunaan jQuery untuk manipulasi DOM. Hasil praktikum menunjukkan bahwa JavaScript modern sangat mendukung pembuatan aplikasi web yang interaktif dan terstruktur.

E. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Modern JavaScript (ES6+) mempermudah pengembangan aplikasi web melalui sintaks yang lebih efisien dan fitur yang lebih modern. Melalui pembuatan webstore sederhana dapat dipahami penerapan langsung JavaScript modern dalam membangun aplikasi web interaktif sebagai dasar pembelajaran React dan framework modern lainnya.

F. Lampiran Repositori GitHub

[Felda-1/Praktikum-Pemrograman-WEB](#)